



SAPIENZA
UNIVERSITÀ DI ROMA

Evaluation of Statistical Methods for Generative Password-guessing models

Facoltà di Ingegneria dell'informazione, informatica e statistica
Informatica

Roberto Di Rosa

ID number 2043388

Advisor

Prof. De Gaspari

Co-Advisor

Academic Year 2025/2026

Evaluation of Statistical Methods for Generative Password-guessing models
Sapienza University of Rome

© 2025 Roberto Di Rosa. All rights reserved

This thesis has been typeset by $\text{\LaTeX}$ and the Sapthesis class.

Author's email: dirosa.2043388@studenti.uniroma1.it

Abstract

"Don't be overwise; fling yourself straight into life, without deliberation; don't be afraid - the flood will bear you to the bank and set you safe on your feet again."
Fyodor Dostoevsky

Contents

1	Introduction	1
2	Motivation	2
2.1		2
3	Background and Related Work	3
3.1	Monte Carlo Estimation	3
3.2	Markov Chains	3
3.3	Recurrent Neural Networks(RNN)	4
3.4	Long Short Term Memory(LSTM)	4
3.5	Probabilistic Password Models	5
3.6	Previous Works	5
3.6.1	Classical Works	5
3.6.2	Modern	6
4	Methodology	7
4.1	Optimizing FLA sampling process	7
4.1.1	Pure Python	7
4.1.2	Heap-item	7
4.1.3	Custom-heap	8
5	Experimental Setup	9
5.1	Dataset	9
5.2	Model and Training	9
5.3	Evaluation Metrics	9
5.3.1	Password Rank Estimation	9
5.3.2	Guess Number and Guessing Percentage	9
5.3.3	Estimation with Monte Carlo	10
5.4	Implementation Details	11
5.4.1	Code	11
6	Results and Analysis	12
6.1	Figures	12
6.2	Tables	13
7	Conclusion	15
7.1		15
.1	UML	15
.2	Extended FOL	16
	Bibliography	17

Chapter 1

Introduction

In the field of probabilistic Password-guessing models... **This thesis makes the following contributions:**

0. A Cython-based item structure that reduces memory usage by 59.79%.
1. A custom-heap implementation that reduces memory usage by 87.47%.
2. Integration into MAYA to accelerate Monte Carlo estimation.
3. Evaluation of accuracy across Θ sizes.
4. A reproducible implementation.

Chapter 2

Motivation

2.1

Chapter 3

Background and Related Work

3.1 Monte Carlo Estimation

Given an analytically complex problem to solve, using the Monte Carlo estimation will give us a value close enough to the real solution, even though it is much more simple analytically speaking. If we take a sample of n items from the main problem, we then perform a sum over the values then divide at the end with *#samples*. This will give us an approximation of the expected value of our original problem. Let us say that we do not know the probability of a coin toss, we can use Monte Carlo to estimate this probability:

Given a Bernoulli random variable $X \in \{0, 1\}$

$$\hat{p} = \frac{1}{n} \sum_{i=1}^n X_i$$

As shown in the formula, above and according to the weak law of large numbers ¹

$$\lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n X_i = 0.5$$

3.2 Markov Chains

Markov chains give us a framework through which we can model sequential events it works like this: we have a set of states for each state we have associated a probability distribution of going to another, that can be easily represented with a matrix. To formalize this we can look at the below representation

Let $S = \{S_1, S_2, \dots, S_n\}$ be the state space.

Let P be the $n \times n$ transition matrix, where

$$P_{ij} = P(X_{t+1} = S_j \mid X_t = S_i)$$

We can see applications of this in fields such as: NLP², PPM³ etc...

¹The weak law of large numbers states that an observed sample average from a sample will be closer to the true population average as the sample grows larger. [3]

²Natural Language Processign

³Probabilistic Password Models

3.3 Recurrent Neural Networks(RNN)

Models that have 3 layers⁴:

0. Input Layer X
1. Hidden Layer H
2. Output Layer Y

This is what happens at each step t : the input vector x_t is used by the RNN to update its hidden state h_t , with this function[5]:

$$h_t = \sigma_h(W_{xh}x_t + W_{hh}h_{t-1} + b_h), \quad (3.1)$$

W_{xh} is the matrix that represents the weights between the input layer and the hidden one[5], W_{hh} is the weight matrix for the recurrent connection and b_h is just the bias vector.

At the end the model will update it is y_t state in this manner [5]:

$$y_t = \sigma_y(W_{hy}h_t + b_y) \quad (3.2)$$

W_{hy} is the matrix that represents the weights between the hidden layer and the output layer[5], and b_y is the bias vector. The recurring part is the fact that each time we are computing the hidden state, however this model type does not have a good "Memory", because during the hidden state calculation the gradient starts to "vanish" which means that the model cannot learn long sequences of words, to solve this problem we have many evolutions of this kind of mode.

3.4 Long Short Term Memory(LSTM)

This is an evolution of RNN's which mitigates the vanishing gradient problem by controlling it. They introduce the concept of these 3 gates:

0. input
1. forget
2. output

Each of these tell the model how much of the previous, input, hidden, and output states to consider from the previous state. Even though we have this improvement over the base model, this is still not perfect as the vanishing can still occur, nevertheless it is better than the normal one[5].

⁴This is a simplification [5]

3.5 Probabilistic Password Models

P.P.Ms are a necessary part of a modern login security system they are needed to evaluate the strength of the user's password, since today's attackers also use these models to crack passwords more efficiently. When defending they evaluate the strength of a given password by telling you how many attempts it would take to guess it this is known as the **Rank** of a password, the higher the rank the more secure the password. Attackers prefer to use these models instead of brute forcing, Brute forcing is a procedure in which we try every possible solution to find it e.g. "What is Roberto Di Rosa's Gmail password?"

Let's assume he used only characters from the english alphabet and that the password has a maximum length of 12 characters, so we know that there are 26 characters in the english alphabet let's start by the first:

0. a
1. b
2. c ...
3. aa
4. ab
5. ac

We will try until we reach our result, the problem with this approach is that we might have to try: $26^{12} \sim 95.43$ quadrillions times which for any pc is very computationally demanding. We could use optimized versions like [John The Ripper](#), which fasten the brute forcing job but are still very slow compared to these models. These probabilistic models instead rely on something else, after training we take all of the tuples of

0. password
1. password's probability

and put them inside of a set Γ , They work by getting trained on a dataset(e.g. RockYou,Libero,TaoBao,etc...), from these datasets they will take the most frequent passwords and they will create distributions for these passwords in a way as to reflect the frequency of characters, all of this using one of these RNN's, Markov chains, PCFGs, LSTMs, Random Forests[6], They can be separated into 2 types:

0. **Trawling** are aimed guessing correctly as many passwords as possible[7] and are constrained just by the amount of resources and the number of guesses itself.
1. **Targeted** are aimed at guessing the password of a single user given some specifics (e.g.: Name,Surname,Sex,Nationality,etc...)

3.6 Previous Works

3.6.1 Classical Works

Fast Accurate And Lean(FLA)

FLA is an LSTM model that is used to guess passwords and study how effective an attacker might be at cracking passwords. As evidenced in [4] the model is

very good at guessing passwords compared to PCFGs, Markov Models and more heuristic methods like John The ripper. Although in recent years there has been a development in to password guessing, as in the emergence of new kinds of model like: RFGuess [6], PASS LLM [7] etc... The authors of FLA have done an excellent job at making it very "lean" so that it can run easily on the client side, making it a really good model for password strength metering on the client-side which was a novel idea at the time. They also used Monte Carlo estimation[2] to efficiently look at how many guesses it would take to guess all of the test dataset [4], and proved reliability up to 10^7 .

3.6.2 Modern

Password Guessing With LLMs

As seen in [7] they brought forward the concept of using LLMs for password guessing and obtained really good results in the evaluation of guesses. These LLMs can get to higher guessing numbers much faster than previous implementations, instead of

Chapter 4

Methodology

4.1 Optimizing FLA sampling process

In this section, we start by analyzing the problems we encountered with MAYA and discussing the solutions we found to the said problems, their strengths, and weaknesses.

The problem we encountered was the need to sample the top-k elements from the distribution within FLA. Solution 1: Implement the item in Cython to improve performance and reduce memory usage. Solution 2: Write all the heap in Cython and do not store any strings.

4.1.1 Pure Python

The first problem we tackled was the original implementation of FLA's sampling algorithm[4], which occupied a significant memory usage due to the usage of pure Python structures such as lists, tuples, strings, and floats. The main problem came from the fact that each item was saved as a tuple of str, float, which was appended into a list, and that list was manipulated with the `heapq` module. Figure 1 illustrates the significant memory consumption caused by the original technique.

4.1.2 Heap-item

The first proposed solution was to build a `heap-item` custom module written in Cython thus maximizing performance and compatibility with Python. The main problem with this approach is the fact that like in pure python, each heap item is an object that must be tracked by python's `garbage collector`, nevertheless this approach helped us reduce memory consumption by $\sim 59.79\%$ instead of the pure python approach as seen in Figure 1, alas further optimization was necessary, so we opted for a custom heap called `heapcy`.

4.1.3 Custom-heap

Rather than simply implementing a heap in Cython, we adopted a different approach by storing only the string's position in a file as a reference. This method significantly reduced memory usage, as shown in Figure 1, and required only a single heap object instead of multiple instances as in previous implementations 4.1.1 and 4.1.2. This approach has given us an improvement of $\sim 87.47\%$ over 4.1.1 and a $\sim 68.85\%$ over 4.1.2. While this solution offers superior performance, it is more complex to use compared to the previous solutions, so the choice should depend on the specific requirements of your situation.

Chapter 5

Experimental Setup

5.1 Dataset

For this paper we chose the [RockYou](#) dataset the same as in MAYA [1](split 80;20), for time reasons we couldn't test with other datasets.

5.2 Model and Training

As discussed, there was no time to implement the functions required to test with other models. Model wise we used FLA found in MAYA[1], and optimized as said in 4.1. The passwords were generated with a maximum length of 12 characters, the model was pre-trained in the RockYou dataset and as a test-config we used [rq1](#).

To evaluate these models we need to see how accurate their guessing [4] is, the following section will show us how to do this in multiple ways.

5.3 Evaluation Metrics

5.3.1 Password Rank Estimation

Why can't we just use the deterministic rank ? While using the deterministic rank would give us the most precise result, it requires an enormous amount of resources since we first need to calculate the entire set of passwords yieldable by the model, so we opt for the estimation As described above, to get our guessing percentage estimate we first need to implement the Monte Carlo seen in [[2]]. To do this we need 1 set:

$\Theta := \{(pwd, prob) \mid (pwd, prob) \in \text{getTheta}(\text{model}, \text{sampleSize})\}$ our Monte Carlo sample.

5.3.2 Guess Number and Guessing Percentage

The standard way to get the guessing percentage of a model is the empirical way:

Eq. Empirical guess number for a given x.

$$G_{\text{emp}}(x) = \frac{|\mathcal{G}_x \cap \mathcal{T}|}{|\mathcal{T}|} \quad (5.1)$$

This will give us the real percentage as seen in [2](#) last column and it will tell us how good the model is in reality. The former approach was very slow, we needed to find another way to reach a good enough solution.

5.3.3 Estimation with Monte Carlo

We employ a Monte Carlo estimate ([5.2](#)) to get a result similar to ([5.1](#)). $\hat{R}$, represents the estimate of the rank (number of times the model needs to guess¹) to generate the password.

Thanks to this we require much less effort compared to the deterministic way, since all we need to do is to compute the Monte Carlo curve just once per Model Checkpoint, save it to .csv file and then for each guess number apply the formula above and we will have our results significantly faster than the previous approach.

The former approach was very slow since we needed to take the top_k where k is the guess number [\[1\]](#).

There are two ways to calculate the Monte Carlo estimation both specified in [\[2\]](#) and here included:

Eq. Monte Carlo Estimation for the rank of a password.

$$\hat{R}_{\Psi}(\alpha) = \hat{\mathbb{E}}_{\beta \sim \Psi} \left[\frac{1[p(\beta) > p(\alpha)]}{p(\beta)} \right] = \frac{1}{n} \sum_{\beta \in \Theta} \frac{1[p(\beta) > p(\alpha)]}{p(\beta)} \quad (5.2)$$

Array sorted in descending order

$$\text{Let } \mathbf{A} = [p(\beta_1), p(\beta_2) \dots, p(\beta_n)] \quad (5.3)$$

Eq. Optimized version

$$\text{Let } \mathbf{C}[i] = \frac{1}{n} \sum_{j=1}^i \frac{1}{A[j]} = C[i-1] + \frac{1}{n \cdot A[i]}$$

Estimation of the guess number for a given x.

$$\hat{G}(x) = \frac{1}{|T|} \sum 1_{[\hat{R}(\alpha) \leq x]} \quad (5.4)$$

Eq. Variant of ([5.4](#))

$$\hat{G}(x) = \frac{|\{\alpha \in \mathcal{T} \mid \hat{R}(\alpha) \leq x\}|}{|\mathcal{T}|} \quad (5.5)$$

¹As defined in [\[2\]](#)

5.4 Implementation Details

5.4.1 Code

```

1 def montecarlo_test(
2     self, eval_dict: dict, iter: Iterable[Tuple[str, torch.Tensor
3     ]]:
4     mc_result_path = f"/tmp/mc_results_{self.n_samples}.csv"
5     A, C = self._build_mc_curve(eval_dict)
6
7     """
8     Searchsorted returns the index immediately left of that
9     insertion point
10    is the last existing element that compares as "less than"
11    the insertion relation.
12    Because we inverted with -A, using right=False ensures
13    idx is the first position
14    where -A >= -target (i.e., A <= target) so idx - 1 is the
15    last position where A > target.
16    """
17    with open(mc_result_path, "wt") as f:
18        w = csv.writer(f)
19        w.writerow(["pwd", "real_rank", "rank_hat", "prob"])
20        for i, (pwd, target) in enumerate(iter):
21            idx = torch.searchsorted(-A, -target, right=False) -
22            1
23            if idx < 0:
24                r_hat = torch.scalar_tensor(0, device=A.device)
25            else:
26                r_hat = C[idx]
27            w.writerow((pwd, i, r_hat.item(), target.item()))

```

Listing 5.1. Code snippet taken from the model.py file, which represents our montecarlo function.

Chapter 6

Results and Analysis

6.1 Figures

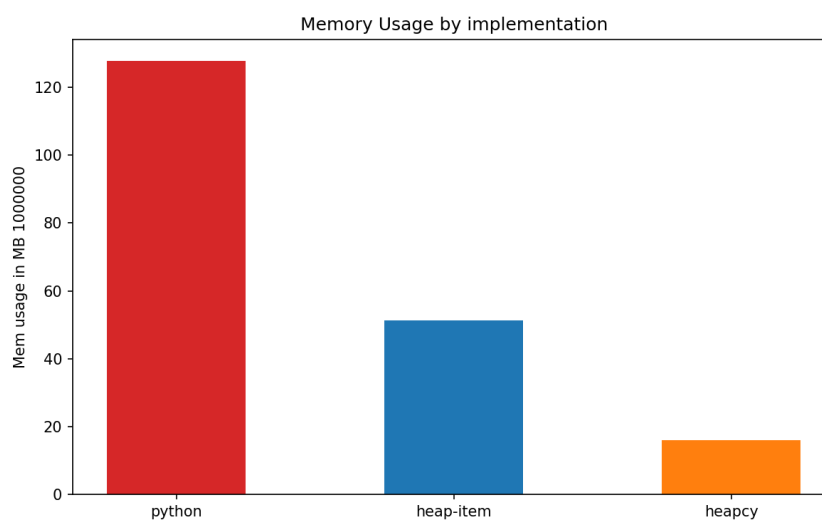


Figure 1. Memory Usage, of each implementation with 10^6 items

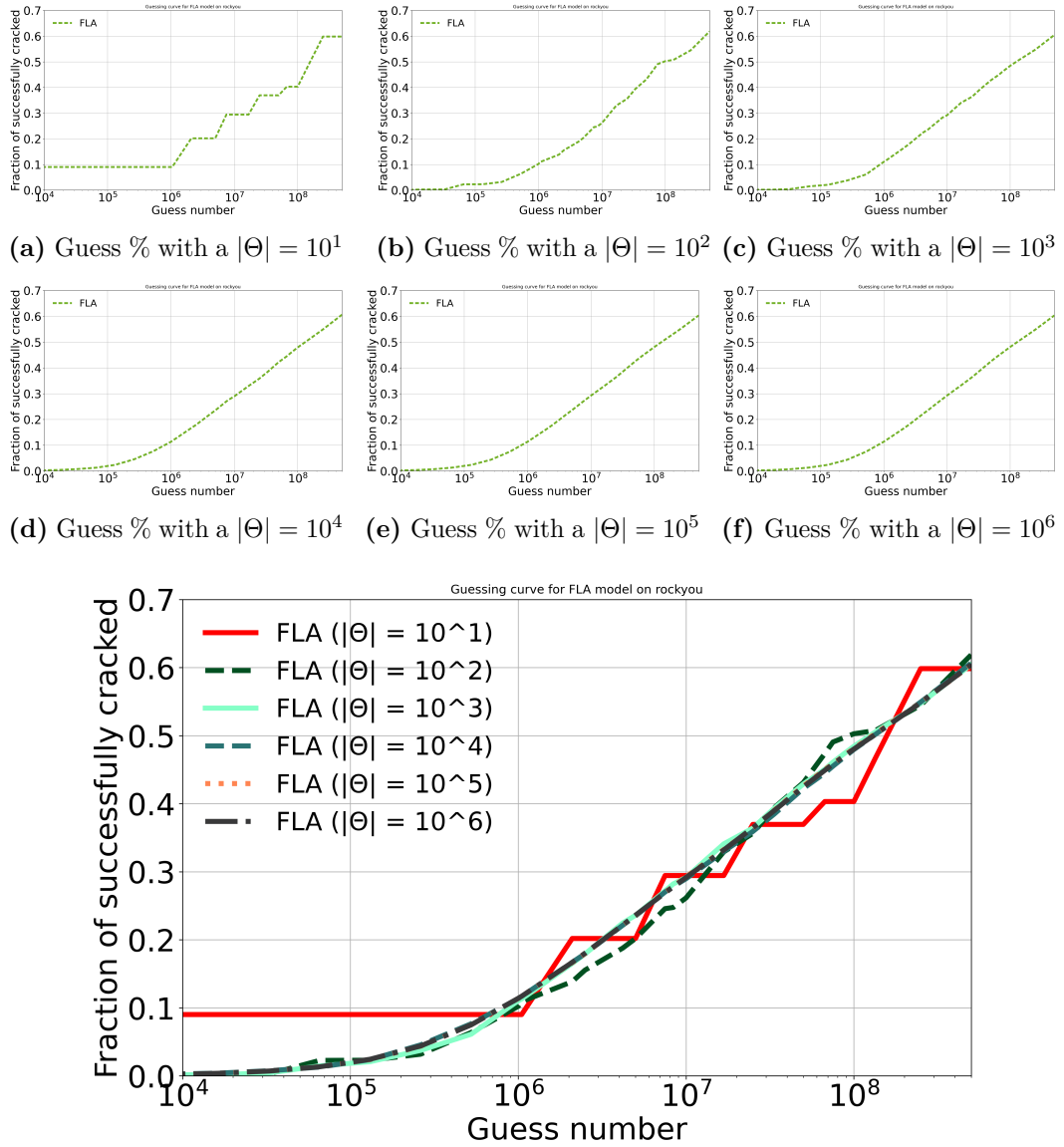


Figure 4. Comparison of the montecarlo run for every theta size

6.2 Tables

#Item	Pure Python	Heap Item	Heapcy
10^6	127.72	51.36	16.00
10^5	12.72	5.13	1.60

Table 1. Size occupied in MB

As can be seen the gains have the same rate at any amount of items, so by using 10x the amount of items we will have 10x the amount of memory used.

Guess #	Rate 10^1	Rate 10^2	Rate 10^3	Rate 10^4	Rate 10^5	Rate 10^6	MAYA %
$1.0 \cdot 10^6$	9.01%	10.22%	11.03%	11.35%	11.37%	11.40%	11.40%
$2.5 \cdot 10^6$	20.19%	15.54%	17.91%	17.95%	17.97%	18.02%	18.04%
$5.0 \cdot 10^6$	20.19%	20.19%	23.60%	23.49%	23.57%	23.61%	23.66%
$7.5 \cdot 10^6$	29.43%	24.57%	27.17%	27.02%	26.94%	26.90%	26.94%
$1.0 \cdot 10^7$	29.43%	26.15%	29.15%	28.97%	29.25%	29.16%	29.20%
$2.5 \cdot 10^7$	36.95%	35.63%	36.41%	35.91%	36.42%	36.45%	36.48%
$5.0 \cdot 10^7$	36.95%	43.09%	42.88%	42.23%	42.59%	42.47%	42.49%
$7.5 \cdot 10^7$	40.31%	49.05%	46.09%	45.50%	45.84%	45.80%	45.82%
$1.0 \cdot 10^8$	40.31%	50.26%	48.43%	47.95%	48.11%	48.02%	48.04%
$2.5 \cdot 10^8$	59.84%	54.45%	54.74%	54.92%	54.85%	54.83%	54.85%
$5.0 \cdot 10^8$	59.84%	61.87%	60.45%	60.65%	60.35%	60.42%	60.47%

Table 2. Comparison of crack rates over five runs with different θ size.

Chapter 7

Conclusion

7.1

In this thesis we evidenced how employing Monte Carlo's methods for the estimation of guess-estimates works up to $5.0 \cdot 10^8$ more than the implementation in [4], since the results are so close to the deterministic one we believe this may indicate that it can be used to estimate the guess rate for even more passwords, for best accuracy we recommend using 10^6 but if you don't have the compute power you can also use 10^4 for a more rough but accurate enough estimation. For future works we could implement this for the other models in MAYA, and push the number of guesses even further (e.g. $1.0 \cdot 10^9$, $2.5 \cdot 10^9$)

Appendix

.1 UML

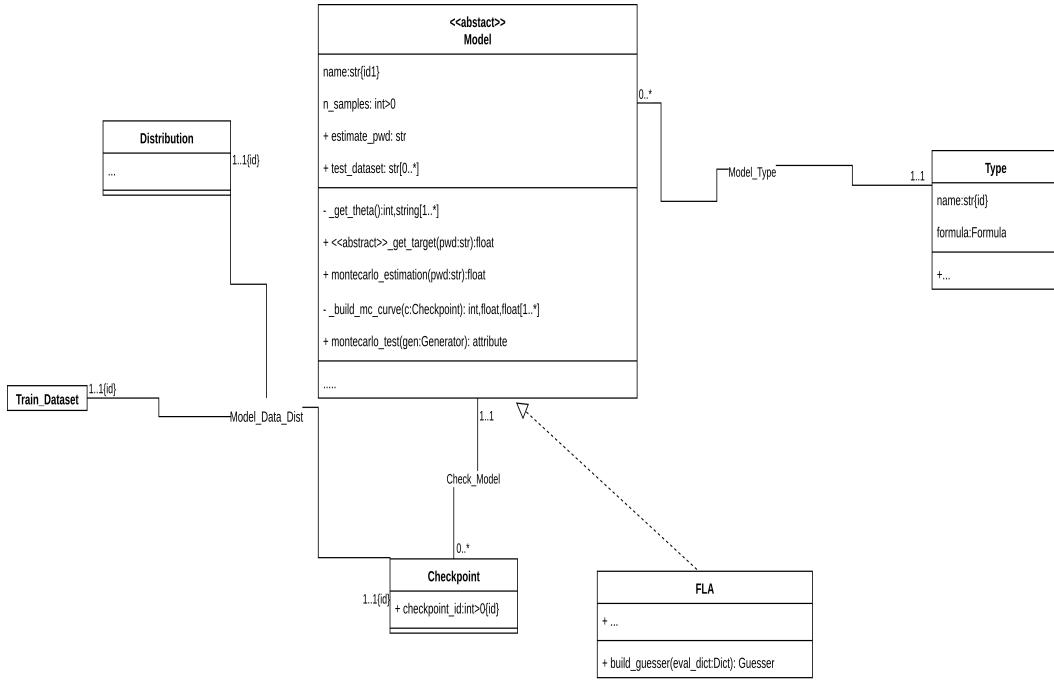


Figure .1. Analysis

.2 Extended FOL

Let $\mathbf{A} := \{(i, prob) \mid i \in \mathbb{N} \wedge (x, prob) \in sorted(\{x, p \mid \exists pwd \in \Theta \wedge x \in \mathbb{N} \wedge P(pwd, p) \wedge x = x + 1\}, sort_func) \wedge i = |\mathbf{A}| + 1\}$ (5.3)

$sort_func(t_1 : int \geq 0, real(0..1), t_2 : int \geq 0, real(0..1)) \rightarrow Bool :$

preconditions : None

postconditions :

$|ret := \forall n_1, n_2,$

$|Real(t_1, n_1) \wedge Real(t_2, n_2) \implies n_1 > n_2$

$|return ret$

Let $\mathbf{C} := \{(i, n) \mid \exists x, prob \wedge i \in \mathbb{N} \wedge n \in \mathbb{R}_{>0} \wedge |A| > 0 \wedge n = \frac{1}{|A|} \cdot \sum_{x, prob \in A}^{i \leq x} \frac{1}{prob}\}$

Bibliography

- [1] CORRIAS, W., DE GASPARI, F., HITAJ, D., AND MANCINI, L. V. MAYA: A unified benchmarking framework for generative password-guessing models. In *2026 IEEE Symposium on Security and Privacy (SP)* (2026). ArXiv preprint arXiv:2504.16651v2. Available from: <https://arxiv.org/abs/2504.16651v2>.
- [2] DELL'AMICO, M. AND FILIPPONE, M. Monte carlo strength evaluation: Fast and reliable password checking. In *Proceedings of the 22nd ACM Conference on Computer and Communications Security (CCS'15)*. ACM Press (2015). Available from: <https://dl.acm.org/doi/pdf/10.1145/2810103.2813631>.
- [3] LOÈVE, M. *Probability Theory I*. Springer, New York, 4th edn. (1977). Available from: https://archive.org/details/probabilitytheor0000loev_h6n0/mode/2up.
- [4] MELICHER, W., UR, B., SEGRETI, S. M., KOMANDURI, S., BAUER, L., CHRISTIN, N., AND CRANOR, L. F. Fast, accurate, and lean: A low-overhead, high-accuracy method for a-priori password strength estimation. In *25th USENIX Security Symposium (USENIX Security 16)* (2016). Available from: <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/melicher>.
- [5] MIENYE, I. D., SWART, T. G., AND OBAIDO, G. Recurrent neural networks: A comprehensive review of architectures, variants, and applications. *Information*, **15** (2024). Available from: <https://www.mdpi.com/2078-2489/15/9/517>.
- [6] WANG, D., ZOU, Y., ZHANG, Z., AND XIU, K. Password guessing using random forest. In *32nd USENIX Security Symposium (USENIX Security 23)* (2023). Available from: <https://www.usenix.org/conference/usenixsecurity23/presentation/wang-ding-password-guessing>.
- [7] ZOU, Y., AN, M., AND WANG, D. Password guessing using large language models. In *USENIX Security Symposium (USENIX Security 25)* (2025). Available from: <https://www.usenix.org/conference/usenixsecurity25/presentation/zou-yunkai>.